3. Run

3.1 Start the GUI: TreeSyncRoot or TreeSyncHome

Two programs sync, with the same window; they differ in who the syncs run as:

	TreeSyncRoot	**TreeSyncHome**
Start it	**TreeSyncRoot-GCC** or **-Clang** in the TreeSyncApp folder, or `treesync-root-gcc`	**TreeSyncHome-GCC** or **-Clang** in the TreeSyncApp folder, or `treesync-home-gcc`
Syncs as	**root**, each run through `pkexec`	you
Paths	any	only inside your home
Who	administrators (root, `wheel`, `sudo`, `admin`); anyone else is told to use TreeSyncHome	anyone; it refuses to start as root
Settings	`~/.config/treesync/settings`	`~/.config/treesync/settings-home`

Each comes once per compiler, GCC and Clang; TreeSyncInstall installs one or both (see 2.6). They do the same; both builds share the settings, database and logs. For everyday syncs use the **installed** ones. The builds in the source tree (`v001/out/gcc/treesync-home`, `treesync-root`) are for working on TreeSync itself; **Run TreeSyncHome-GCC (just built)** in TreeSyncBuild starts one. Neither has anything to do with building: that is **TreeSyncBuild** (see 2.3).

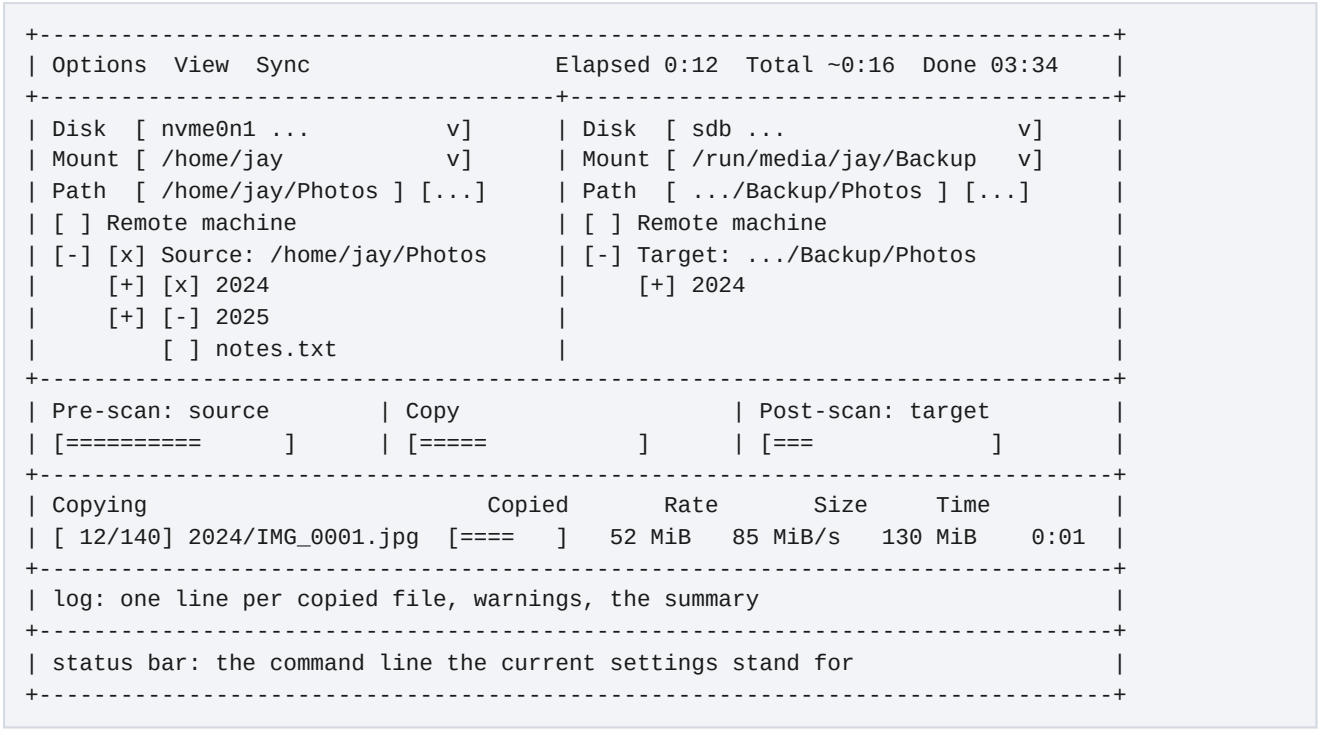
Always start the GUI as yourself, not with `sudo`: it is an ordinary desktop program, and only TreeSyncRoot's syncs run as root.

Options on the command line set up the window and become the saved settings:

```
treesync-root-gcc --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror
```

Without options, the settings saved last time are used (each program its own file, see the table; saved on every change).

3.2 The main window



- **Left: the source**, **right: the target**. Each side has a **Disk** and **Mount** list to jump to a drive, a **Path** box (type a path and press Enter, or **...** to browse), **Remote machine** for a tree on another machine (3.10), and the tree below it. TreeSyncRoot lists every disk; TreeSyncHome lists only your home (the title bar says which window this is).
- **Trees**: click [+] to open a folder, or use the + and - keys. **View > Reload trees** reads them again, **View > Collapse all** closes them.

- **Right-click** a file or folder in either tree to change it by hand. The window then reads the trees again:

    - **New folder...**: a folder inside the one you clicked (or next to the file you clicked).
    - **Rename...**: a new name in the same folder. It never replaces something that is already there.
    - **Delete...**: moves it to the `.old` folder, like the files a sync deletes. It can be brought back from there until **Clear .old**.
    - **Delete for good...**: deletes it, and for a folder everything in it. This cannot be undone.

    The changes are made the way a sync is: as root in TreeSyncRoot (with the same one password per window), as you inside your home in TreeSyncHome, over SFTP for a remote tree. They are logged like a run (`Edit (delete)` in Results.txt). The menu is greyed out while a sync runs, and TreeSync's own files (`.treesync-old`, `*.treesync-part`) and the top folder itself cannot be changed. Changing the **source** also changes what the next run does: a file deleted from the source is removed from the target by the next Mirror. - **Free space**: under each tree, a bar and a line show the disk's free space, e.g. `384.0 GiB free of 930.0 GiB (58% used)`. It turns red under 5% free. Local trees show it all the time. A remote tree shows it once a run has asked its server. During a run it updates after every file, at most once a second. Between the disk's own figures, at the start and the end of the run, TreeSync works it out from what the copies write (holes count as nothing), shown with a `~`. So another program filling the disk at the same time only shows at the end. - **The progress boxes** and the list of files in progress appear while a run is going. Each file in progress

shows a bar, **Copied** (the bytes copied so far; **Read** in a Scan only run), its **Rate**, its **Size** and its **Time** so far. When idle threads help with a big file (*Split big files across idle threads*), its row counts what all of them copied, and the log says *splitting NAME*. The upper right of the menu bar shows how long it has run, how long the whole run will probably take, and when it should be done. - **The log** shows what happened; **View > Show log file** (Ctrl+L) opens the full log of the newest run.

Without root, local paths are limited to your home folder; the status bar says so.

Help opens this manual: the whole of it, or one chapter, as a PDF. **F1** opens the chapter for the window you are in.

3.3 Choose what to sync: the checkboxes

Every folder and file in the **source** tree has a checkbox.

- **Check a folder**: everything below it is synced, including files added there later.
- **Uncheck a folder**: nothing below it is synced.
- **Uncheck one file** inside a checked folder: the rest of that folder stays checked.
- A folder with only some of its contents checked shows a **dash** (partly checked).
- The **top box** checks or unchecks the whole tree.

What is checked is saved as `--include-path` settings. Folders above a checked item are created on the target only where something checked goes, and **Mirror never deletes anything outside the checked paths**. Changing the source folder starts again with everything checked. If nothing is checked, Run refuses to start.

3.4 Options: mode and switches

Options (in the menu bar) opens the sync's mode and switches. **OK** keeps the changes, **Cancel** throws them away. The trees and their remote logins are not here: they are on the main window (3.2, 3.10).

- **Mode**:
  - **Copy**: new and changed files go from source to target; nothing on the target is removed. Turns on *Skip older*: a newer file on the target is not overwritten.
  - **Mirror**: the target is made to match the source; files only the target has are removed (with safe delete: moved to `<target>.old`, or to `<target>/.treesync-old` when the target is the top folder of a disk, such as a USB drive).
  - **Merge**: changes go both ways; a file changed on both sides since the last sync is a conflict and is left alone. Nothing is removed.
- **Reset to the defaults** of Copy, Mirror or Merge.
- **Owners**: don't copy / by name (normal) / by number. Setting an owner needs root.
- **Switches**: click any option to see what it does in the help box. The important ones: *Safe delete* (on; keep it on), *Hard links*, *Follow links*, *Skip older / newer / changed*, *Keep extra*, *Wipe target first*, *Check there is room before copying* (on; see 3.5), *Low priority*, *Split big files across idle threads* (on: a thread with no file left to start copies chunks of a big file another thread is on, so the last big file of a run is not left to one slow stream).
- **Numbers**: threads (default 4), retries, chunk size, bandwidth limit, how long to wait for a lost network, time tolerance.

- **Include / exclude**: file and folder names with `*` and `?`, separated by `;`. A blank box filters nothing; exclude wins over include.

3.5 Dry run, then run

Room check first. Before a run copies anything, it lists both trees (quickly, without reading files) and adds up the most the copies can need: new and changed files at their full size. If that, plus a reserve, is more than the target disk has free, the run does not start (exit code 4) and the GUI shows the numbers and asks whether to start anyway. A dry run shows the same check, in the log and in its popup at the end (green, or red when it may not fit), e.g. *room check: the copies need at most 2.1 GiB (14 files); the target disk has 457.1 GiB free: room to copy*.

The check cannot be exact. A file with long runs of zeros (a VM image) takes much less disk than its size: the 400 GiB image above takes 53 GiB. That is only known when it is read, so **start anyway** is often right for VM images. Either way, a run that gets close to filling the target **stops by itself**, before the disk is full in the middle of a file. It keeps a reserve free: 1% of the disk, at least 256 MiB (or 5% of a disk too small for that), at most 4 GiB. The log says *stopping: the target disk is nearly full*, and once you have made room, **Run** continues where it stopped. The switch **Check there is room before copying** (`--no-precheck`) turns the check off; the stop near full stays on.

1. **Sync > Dry run** (Shift+F9): lists what a run would do (`copy`, `link`, `delete`, `attrs`, `mkdir`) and changes nothing. Read it.
2. **Sync > Run** (F9): asks for confirmation, saying what the mode will do and where removed and replaced files go, then runs.

While it runs:

- **Pre-scan: source** walks the source. **Copy** shows the bytes copied of the total. **Post-scan: target** checks each target folder for files to remove (Mirror).
- Each file is written to `NAME.treesync-part`, read back, checked against the source's SHA-256, given the source's permissions, times, owner and extended attributes, and only then renamed into place.
- A file whose content is the same but whose permissions or time changed gets only its attributes set (an `attrs` step), without copying.

At the end a popup says how it went and how long it took. **Sync > Stop** (Esc) stops after the current file or chunk.

When files were copied, the popup and the end of the log also show the **check**: the numbers that prove the copies are the source.

```
check: 1 file copied and read back; its SHA-256 matches the source's
  bytes: 429,562,527,744 on the source, 429,562,527,744 in the copies: the same
  this run copied 381.3 GiB; the other 18.8 GiB (...) were copied by the stopped run
    before it, read back again and kept
  on disk the copies take 53.0 GiB for 400.1 GiB of data: runs of zeros are kept as
    holes (sparse), which read back as zeros, so the content and the SHA-256 are the same
  mode, time and owner: kept
  the file:
    Fedora_Base.qcow2
      source  429,562,527,744 bytes  SHA-256 0256ebac...
      copy    429,562,527,744 bytes  SHA-256 0256ebac...  MATCH
      time    2026-09-19 23:38:18.273011253 on both
      mode    600 on both;  owner root:libvirt on both
```

Every byte is counted on both sides, and the SHA-256 of each copy (read back from the target) is shown next to the source's. With many files, the totals come first, then the 10 biggest in full. A number that differs on purpose says why where it is shown: - **Resumed run:** it copies fewer bytes than the file has, because a stopped run had already copied the rest, and that part was checked again. - **Holes:** a copy can take less disk than its size (`du` shows less than `ls -l`). Runs of zeros are kept as holes. Results.txt gets the totals on the run's line, e.g. *1 file checked: 429,562,527,744 bytes on both sides, SHA-256 match*.

3.6 Stop and resume

A stopped run (Stop, Ctrl+C in a terminal, a crash, a reboot, an unplugged disk) is **resumed by running it again with the same settings**. Every step was recorded in the database, so it continues where it was. A big file continues at its last good chunk: on resume, every chunk already copied is read back and checked first, and from the first damaged one the file is copied again.

To start over instead, use `--fresh` on the command line.

3.7 Checking and attributes

- **Sync > Scan only (SHA-256 check)** (F7): changes nothing. Lists both trees, reads and hashes **every file on both sides** afresh, and compares permissions, times, owners and extended attributes. Each file's log line ends in `same`, `DIFFERS`, `missing` (not on the target) or `failed`. Use it after a big copy to prove the target is right.
- **Sync > Sync attributes** (F8): sets only the attributes that differ (mode, time, owner, extended attributes) on the target, for files of the same size and for folders. No content is copied.
- **Sync > Clear .old folders...**: deletes the safe-delete folders (`<target>.old`) for good, after a confirmation that says how many files and bytes. Folders made by a root run need **TreeSyncRoot**. On the command line, without `--no-dry-run` it only counts; its result says *Counted, not deleted*, *Good* (deleted) or *Nothing to clear*.

3.8 History, logs and AI export

- **View > History...** (Ctrl+H): every run, newest first, with its counts (folders, copied, deleted, attributes set, failed, conflicts, bytes). Failed runs are red. Below:
  - **Actions**: what the selected run did, failures first.

    - **SHA-256 checks**: every checked file under a line naming its folder, with the source and target SHA-256 side by side, whether they match, and how the attributes compare. Mismatches first, in red; attribute differences in orange.
    - **Show log**, **Export logs to AI.md**, **Export SQL to AI.md**.
- **Logs**: each run writes `logs/YYYY-MM-DD_HHMMSS.log` next to the database. It starts with a header; every line has its start, stop and duration to the microsecond.
- **View > Export newest run for AI**: writes the run as Markdown files into `logs/ai/` (a guide, problems, slowest copies, throughput, the logs in full; or the database contents), ready to give to an AI to analyse.

3.9 Running as root

A sync often meets files your user cannot read or replace (other users' files, system files), so **TreeSyncRoot syncs as root, for an administrator**: root, or a member of `wheel` (`sudo` or `admin` on Debian, Ubuntu and Mint), the users who may use sudo. A tool that can overwrite any file as root is as powerful as root, so nobody else gets that: for anyone else TreeSyncRoot says so and does not start, and they use **TreeSyncHome**, which syncs as themselves, inside their own home, with no password asked.

The window's title says which one it is and how its syncs run: *TreeSyncRoot-GCC - syncs as root*, or *TreeSyncHome-GCC - syncs as jay, inside /home/jay*. TreeSyncRoot offers every disk; TreeSyncHome only your home.

For an administrator:

- **TreeSyncRoot** runs every sync through `pkexec`, with the installed `treesync-cli-gcc` (or `-clang`: the one of its own compiler), which only root can change. The first run of a window asks for your password, once: that window's later runs go through the same root process (`treesync-cli --root-session`), which ends when the window closes. A window opened within a few minutes does not ask either (polkit remembers it). Any path may be chosen.
- **TreeSyncHome** runs every sync as you (it passes `--home` to `treesync-cli`), so it never asks for a password and stays inside your home.
- **The installed `treesync-cli`**, started from a terminal by an administrator, starts itself again through `sudo`: `treesync-cli --source=... --target=...` runs as root (sudo asks for your password). `--help` and `--home` do not.
- **The source-tree TreeSyncRoot** (`out/gcc/treesync-root`) also uses the **installed** `treesync-cli`. If that is not this build, the log says so (TreeSyncInstall updates it); if nothing is installed, it warns that it runs a copy your user could change.

A root run keeps its own database and logs under `/root/.local/share/treesync/`, separate from yours. To resume a root run, run it as root again.

History and logs of root runs. The GUI's **History** and **Show log file** read *your* database and logs, and root's are not readable by you. To look at a root run: `sudo ls /root/.local/share/treesync/logs/`, or have root export it: `treesync-cli --export-ai-logs` (the installed one runs as root by itself; it writes into root's `logs/ai/`). Note that `sudo treesync-cli` does not work on RHEL: sudo does not search `/usr/local/bin`; the installed `treesync-cli` needs no sudo in front.

3.10 Remote trees (SFTP)

Either side can be on another machine over SSH. One-time setup:

```
ssh-keygen -t ed25519            # if you have no key yet
ssh-copy-id jay@server           # passwordless login
ssh jay@server                   # once, to check and accept the host key
```

TreeSync logs in with keys only, never passwords. In the GUI, on the side that is remote (usually the target), tick **Remote machine** under Path. A login row opens:

```
[x] Remote machine
Host [ server              ]  Port [22]  User [ jay  ]
Key  [                     ]          [Test]  [Done]
```

Fill in Host, Port and User (and Key, if not the usual one), and the Path above it (**...** picks it from the server's folders, starting at `/`). **Test** logs in, lists the path and, when that works, uses the login and folds the row into one line:

```
[x] Remote machine   jay@server:22   (test passed: 12 entries)   [Show / Test]
```

If the test fails it says why and leaves the row open. **Done** uses the login without a test. **Show / Test** opens the row again, to see it, change it or test it again. Unticking **Remote machine** makes the tree local again at once. Until a login is used (Test or Done), the tree and the runs keep the one they had. On the command line: `--target-host=server --target-user=jay`.

If the network drops, the run waits (default up to 30 minutes, *Wait for the network*), shows **NETWORK UNAVAILABLE**, and carries on when the machine is back.

Checking copies on the server. When the login may run commands and the server has `sha256sum`, the server hashes its own files: copies are checked there and remote scans hash there, so nothing is read back over the network. The log says which way it works (`server hashes its own files` or `files are read over SFTP`). An SFTP-only account still works, just with twice the traffic for a verified copy.

3.11 The command line

`treesync-cli` takes the same options the GUI saves. **It does a dry run unless you give `--no-dry-run`.** The examples use the installed `treesync-cli` (it runs as root); in the source tree it is `v001/out/gcc/treesync-cli` (it runs as you).

```
# What would a mirror do?
treesync-cli --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror

# Do it
treesync-cli --source=/home/jay/Photos --target=/run/media/jay/Backup/Photos --mirror --no-
dry-run

# Copy only new and changed files, 8 threads, at low priority
treesync-cli --source=/data --target=/backup/data --copy --threads=8 --low-priority --no-dry-
run

# Only one folder and one file of the source
treesync-cli --source=/data --target=/backup/data --include-path=projects/2026 --include-
path=notes.txt --no-dry-run

# To a remote machine
treesync-cli --source=/data --target-host=server --target-user=jay --target=/backup/data --
no-dry-run

# Prove a copy: hash both sides, change nothing
treesync-cli --source=/data --target=/backup/data --scan-only

# Set only differing permissions and times
treesync-cli --source=/data --target=/backup/data --sync-attrs --no-dry-run

# Delete <target>.old for good (without --no-dry-run it only counts)
treesync-cli --source=/data --target=/backup/data --clear-old --no-dry-run

# One change by hand (what the right-click menu does): mkdir, rename, delete
# (to the .old folder) or delete-for-good
treesync-cli --source=/data --target=/backup/data --edit=rename --edit-side=target \
             --edit-path=old/name.txt --edit-name=new-name.txt
```

`treesync-cli --help` lists every option. In a terminal it shows one line per copied file, `[ 6/90] name 100% | 85.3 MiB/s | 130.9 MiB | 00m01s`, and Ctrl+C stops it (press twice to quit at once).

Exit codes: 0 in sync (or, for Scan only, the trees are the same); 1 failures, conflicts or differences; 2 could not run (bad options, a path refused); 3 stopped (also by itself, when the target disk was nearly full); 4 may not fit (the room check; nothing was copied, `--no-precheck` starts anyway).

Database views for looking at results directly:

```
sqlite3 ~/.local/share/treesync/treesync.db
```

```
SELECT run, started, state, copied, failed FROM run_summary ORDER BY run DESC LIMIT 10;
SELECT * FROM failures WHERE run = 42;
SELECT path, file_name, problem FROM hash_mismatches WHERE run = 42;
SELECT path, file_name, attr_diff, result FROM attr_mismatches WHERE run = 42;
```